

```
"""
```

```
Yuk reestri bot – foydalanuvchi yozgan erkin matnni (masalan:  
"Услугачдан Тошкентга lift 22 тонна 3 та тент керак нақд") shablon  
matnga aylantiradi va tasdiqlagach kanalga post qiladi.
```

```
Ishga tushirish:
```

```
pip install -r requirements.txt
```

```
BOT_TOKEN=... CHANNEL_ID=... ANTHROPIC_API_KEY=... python bot.py
```

```
ANTHROPIC_API_KEY ixtiyoriy: bo'lmasa, bot oddiy (regex) usulda  
ajratadi – sifat pastroq, lekin ishlayveradi.
```

```
"""
```

```
import json
```

```
import logging
```

```
import os
```

```
import re
```

```
from telegram import (
```

```
    InlineKeyboardButton,
```

```
    InlineKeyboardMarkup,
```

```
    Update,
```

```
)
```

```
from telegram.ext import (
```

```
    Application,
```

```
    CallbackQueryHandler,
```

```
    CommandHandler,
```

```
    ContextTypes,
```

```
    MessageHandler,
```

```
    filters,
```

```
)
```

```
logging.basicConfig(
```

```
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
```

```
    level=logging.INFO,
```

```
)
```

```
log = logging.getLogger("yuk-bot")
```

```
BOT_TOKEN = os.environ.get("BOT_TOKEN", "")
```

```
CHANNEL_ID = os.environ.get("CHANNEL_ID", "")
```

```
ANTHROPIC_API_KEY = os.environ.get("ANTHROPIC_API_KEY", "")
```

```
# Per-chat pending template, waiting for confirm/cancel button press.
```

```
PENDING: dict[int, str] = {}
```

```
# -----  
# Parsing  
# -----
```

```
PARSE_PROMPT = """Sen O'zbekistondagi yuk tashish brokerisan. Quyidagi xabar \n  
- kerakli mashina yoki yukni tashish haqidagi so'rov (zapros), so'zlashuv \n  
uslubidagi o'zbek/rus aralash matn. Undan quyidagi maydonlarni ol va \n  
FAQAT JSON obyekt qaytar, boshqa hech qanday matn yozma:
```

```
{{"yuklash": string yoki null, "manzil": string yoki null, "yuk_turi": string yok  
"mashina_turi": string yoki null, "soni": number yoki null, "dona": number yoki n  
"tonna": number yoki null, "kub": number yoki null, "tolov": string yoki null}}
```

MUHIM qoidalar:

- soni – FAQAT kerakli MASHINALAR soni (masalan "3 ta", "2 mashina kerak"). \n
Agar matnda faqat tovar/buyum soni bo'lsa ("500 шт", "500 dona") – buni \n
"soni"ga QO'YMA, "dona" maydoniga yoz.
- dona – yuk ichidagi buyumlar/birliklar soni, mashina bilan aralashtirma.
- yuklash/manzil – shahar yoki joy nomi. Aniq tanib bo'lmasa, matndagi asl \n
yozilishini o'zgartirmay qaytar – taxminiy transliteratsiya qilma.
- mashina_turi – kerakli kuzov turi (masalan "тент", "изотерма", "рефрижератор"),
faqat aniq aytilgan bo'lsa.
- tonna/kub – raqam sifatida.
- tolov – to'lov turi, qisqa ("накд", "ўтказма"), bo'lmasa null.

MUHIM: yuklash, manzil, yuk_turi, mashina_turi, tolov – bu matn maydonlarining \n
barchasini FAQAT KIRILL alifbosida qaytar, hatto foydalanuvchi lotin yozuvida \n
yozgan bo'lsa ham (masalan "Toshkent" -> "Тошкент"). Aniq tanib bo'lmasa, \n
mazmunini o'zgartirmasdan faqat harflarini kirillga almashtirib yoz.
Noaniq maydonni taxmin qilma – null qoldir.

```
Matn: "{text}"  
"""
```

```
def fallback_parse(text: str) -> dict:  
    """Oddiy regex asosidagi zaxira parser (AI kaliti bo'lmasa)."""  
  
    def find(pattern):  
        m = re.search(pattern, text, re.IGNORECASE)  
        return m.group(1) if m else None  
  
    soni = find(r"(\d+)\s*(?:ta|ta)\b")  
    tonna = find(r"(\d+(?:[.,]\d+)?)\s*тонн")  
    kub = find(r"(\d+(?:[.,]\d+)?)\s*куб")
```

```

dona = find(r"(\d+)\s*(?:шт|дона|dona)\b")
tolov = None
if re.search(r"накд|нақд|naqd", text, re.IGNORECASE):
    tolov = "нақд"
elif re.search(r"перечисл|o'tkazma|otkazma|ўтказма", text, re.IGNORECASE):
    tolov = "ўтказма"

return {
    "yuklash": None,
    "manzil": None,
    "yuk_turi": None,
    "mashina_turi": None,
    "soni": int(soni) if soni else None,
    "dona": int(dona) if dona else None,
    "tonna": float(tonna.replace(",", ".")) if tonna else None,
    "kub": float(kub.replace(",", ".")) if kub else None,
    "tolov": tolov,
}

def parse_with_claude(text: str) -> dict:
    from anthropic import Anthropic

    client = Anthropic(api_key=ANTHROPIC_API_KEY)
    resp = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=400,
        messages=[{"role": "user", "content": PARSE_PROMPT.format(text=text)}],
    )
    raw = "".join(b.text for b in resp.content if b.type == "text")
    match = re.search(r"\{.*\}", raw, re.DOTALL)
    if not match:
        raise ValueError("JSON topilmadi")
    return json.loads(match.group(0))

def cap(s):
    if not s:
        return s
    s = str(s).strip()
    return s[0].upper() + s[1:]

def format_template(p: dict) -> str:
    lines = []
    route = f"{cap(p.get('yuklash')) or '?'} \u2192 {cap(p.get('manzil')) or '?'}

```

```

lines.append(f"\U0001F4CD {route}")
if p.get("yuk_turi"):
    lines.append(f"\U0001F4E6 \u042E\u043A: {p['yuk_turi']}")
if p.get("dona"):
    lines.append(f"\U0001F522 \u041C\u0438\u049B\u0434\u043E\u0440: {p['dona']
bits = []
if p.get("soni"):
    mt = p.get("mashina_turi")
    bits.append(f"{p['soni']} \u0442\u0430 {mt if mt else '\u043C\u0430\u0448\u0438\u043D\u0430'}")
elif p.get("mashina_turi"):
    bits.append(cap(p["mashina_turi"]))
if p.get("tonna"):
    bits.append(f"{p['tonna']} \u0442\u043E\u043D\u043D\u0430")
if p.get("kub"):
    bits.append(f"{p['kub']} \u043A\u0443\u0431")
if bits:
    lines.append(f"\U0001F69B {' '.join(bits)}")
if p.get("tolov"):
    lines.append(f"\U0001F4B0 \u0422\u045E\u043B\u043E\u0432: {p['tolov']}")
return "\n".join(lines)

# -----
# Handlers
# -----

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "Salom! Menga yuk/mashina haqida erkin matn yozing - "
        "shablon qilib, tasdiqlashingizdan so'ng kanalga joylayman.\n\n"
        "Masalan: Улугчатдан Тошкентга 3 та тент 22 тонна нақд"
    )

async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text.strip()
    if not text:
        return

    thinking = await update.message.reply_text("Ishlanmoqda...")

    try:
        if ANTHROPIC_API_KEY:
            parsed = parse_with_claude(text)
        else:
            parsed = fallback_parse(text)

```

```

except Exception as exc: # noqa: BLE001
    log.exception("parse xatosi")
    parsed = fallback_parse(text)

template = format_template(parsed)
chat_id = update.effective_chat.id
PENDING[chat_id] = template

keyboard = InlineKeyboardMarkup(
    [
        [
            InlineKeyboardButton("\u2705 Kanalga yuborish", callback_data="se
            InlineKeyboardButton("\u274C Bekor qilish", callback_data="cancel
        ]
    ]
)
await thinking.edit_text(
    f"Shablon tayyor:\n\n{template}\n\nYuborilsinmi?",
    reply_markup=keyboard,
)

async def handle_button(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    chat_id = update.effective_chat.id
    template = PENDING.pop(chat_id, None)

    if query.data == "cancel" or not template:
        await query.edit_message_text("Bekor qilindi.")
        return

    if not CHANNEL_ID:
        await query.edit_message_text(
            "CHANNEL_ID sozlanmagan – kanal ID'sini muhit o'zgaruvchisiga qo'shin
        )
        return

    try:
        await context.bot.send_message(chat_id=CHANNEL_ID, text=template)
        await query.edit_message_text(f"Yuborildi:\n\n{template}")
    except Exception as exc: # noqa: BLE001
        log.exception("kanalga yuborishda xato")
        await query.edit_message_text(
            f"Yuborib bo'lmadi: {exc}\n\nBot kanalga admin qilib qo'shilganini te
        )

```

```
def main():
    if not BOT_TOKEN:
        raise SystemExit("BOT_TOKEN muhit o'zgaruvchisi kerak")

    app = Application.builder().token(BOT_TOKEN).build()
    app.add_handler(CommandHandler("start", start))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, handle_message))
    app.add_handler(CallbackQueryHandler(handle_button))

    log.info("Bot ishga tushdi")
    app.run_polling()

if __name__ == "__main__":
    main()
```